Manifiesto del Testing

Cada slide compara cómo se ve el testing en ágil contra el enfoque tradicional:

Ágil (lo que se valora)	Tradicional
Testing es una **actividad** que el tester hace dentro del tablero del equipo	Testing es una **fase**, con su propia columna al final
Prevenir bugs, preguntando antes "¿qué pasa si…? ¿por qué? ¿cuándo?"	**Encontrar** bugs cuando ya están hechos
Ser **tester**: entender qué se prueba y cómo	Ser **checker**: tildar si funciona o no
Ayudar a **construir el mejor sistema**	**Romper** el sistema
Todo el equipo es responsable de la calidad	El **tester** es el único responsable y actúa como "puerta de calidad" al final

Tradicional vs Ágil

- **Tradicional (cascada):** requerimientos → diseño → desarrollo → test → deploy, todo en secuencia. El testing aparece recién cerca del release.
- **Ágil:** el mismo ciclo se repite en cada iteración, que dura días y no meses ni años. Codificar, corregir bugs y probar ocurren en paralelo dentro de la iteración.

Los 9 principios

1. **El testing se mueve hacia adelante:** se prueba desde el inicio del proyecto, no al final.
2. **Testing no es una fase:** se hace todo el tiempo.
3. **Todos hacen testing:** developers, PO y tester.
4. **Reducir la latencia del feedback:** enterarse rápido de que algo está mal.
5. **Las pruebas representan expectativas:** un test dice qué espera el cliente.
6. **Código limpio y defectos corregidos rápido:** no se acumulan bugs.
7. **Menos documentación de pruebas:** solo la necesaria.
8. **Las pruebas son parte del "done":** una historia sin probar no está terminada.
9. **De probar al final a ser conducido por pruebas:** primero el test, después el código.

Las 6 prácticas concretas

1. Pruebas de unidad e integración automatizadas.
2. **TDD:** el developer escribe el test unitario antes que el código.
3. Pruebas de regresión a nivel sistema automatizadas.
4. **ATDD (desarrollo conducido por pruebas de aceptación):** igual que TDD, pero a nivel de la historia de usuario. El diagrama de la slide 16 es un ciclo:
    - Una característica con ejemplos genera la historia de usuario.
    - De la historia salen las pruebas de aceptación de alto nivel.
    - Esas pruebas se automatizan, se programan y se ejecutan.
    - Se corrigen los defectos y la historia queda aceptada.
5. Pruebas exploratorias: un tester humano investiga sin guion.
6. Control de versión de las pruebas junto con el código, en el mismo repositorio.

Acá va un solo ejemplo, el cajero con la historia “Como cliente quiero extraer dinero”, con una línea por práctica:

1. **Pruebas de unidad e integración automatizadas:** se automatizan pruebas como validarMultiplo(1500) → falso (unidad) y “el cajero descuenta el saldo en la base de datos” (integración). Corren solas cada vez que alguien sube código.
2. **TDD:** el developer escribe primero el test validarMultiplo(1500) → falso, que falla (Red). Después programa la validación y el test pasa (Green). Por último mejora el código (Refactor).
3. **Pruebas de regresión a nivel sistema automatizadas:** cuando el sprint siguiente se agrega “transferencias”, se vuelven a correr automáticamente todas las pruebas de extracción para confirmar que no se rompió nada.
4. **ATDD:** antes de programar, el PO, el tester y los developers acuerdan las pruebas de aceptación de la historia:
    - Saldo $10.000, extraigo $3.000: el saldo queda en $7.000.
    - Saldo $2.000, extraigo $5.000: mensaje “Saldo insuficiente”.

    La historia se acepta cuando esas pruebas pasan.
5. **Pruebas exploratorias:** el tester prueba sin guion cosas como extraer $0, un monto negativo o todo el saldo, y encuentra que $0 no da error.
6. **Control de versión de las pruebas con el código:** todas las pruebas anteriores se guardan en el mismo repositorio que el código del cajero, así cada versión del código tiene sus pruebas.

Orden en que ocurren en la práctica: 4 (se acuerda qué probar) → 2 (se programa con TDD) → 1 (las pruebas quedan automatizadas) → 6 (se guardan junto al código) → 5 (exploración) → 3 (regresión en los sprints siguientes).

Rol del tester

El tester queda en el centro y trabaja con tres actores:

- **Con el PO:** revisa las historias, define criterios de aceptación, ayuda a entender el negocio e identifica dependencias.
- **Con los developers:** define la estrategia de pruebas, automatiza (ATDD), notifica bugs, da feedback temprano y sugiere mejoras de usabilidad.
- **Con el system team:** prepara la documentación de lo que se integra, difunde los casos y datos de prueba de integración y le comunica los bugs al PO y a los developers.
- **Tareas propias:** prepara el ambiente de pruebas, arma casos y datos del sprint, valida el “Done”, acepta la historia junto con el PO y lleva métricas de calidad.

Cambios culturales

Pasar a testing ágil es más un cambio cultural que técnico. Los desafíos son tres:

- **Cultura organizacional:** filosofía de calidad, ritmo sostenible, empoderar al equipo.
- **Gestión del cambio:** paciencia, ganar credibilidad, celebrar logros. También superar barreras como la pérdida de identidad del tester, la falta de capacitación o no entender lo ágil.
- **Manejo de expectativas** de la gerencia.

Cuadrantes del Testing Ágil

Clasifican las pruebas según dos ejes:

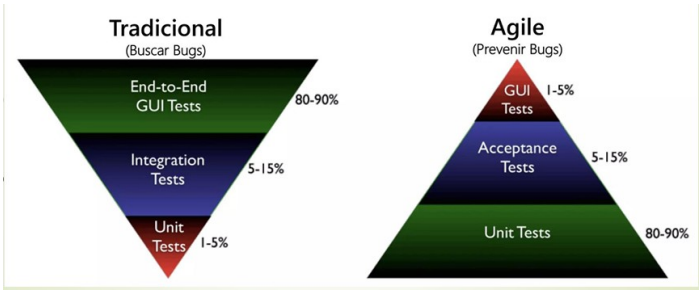
- **Eje vertical:** orientadas al **negocio** (arriba) u orientadas a la **tecnología** (abajo).
- **Eje horizontal: guían el desarrollo** (izquierda) o **critican el producto** (derecha), es decir, lo evalúan una vez hecho.



Pirámide de testing

Qué pasa con cada forma:

- **Invertida (tradicional):** casi todo se prueba al final, por pantalla y con el sistema terminado. Las pruebas son lentas, caras, a menudo manuales y se rompen fácil. Los bugs aparecen tarde, cuando arreglarlos sale más caro.
- **Normal (ágil):** la base es una gran cantidad de pruebas unitarias automáticas que corren en segundos cada vez que se toca el código. Un error se detecta en el momento en que se escribe, así que es una pirámide para **prevenir bugs**. La pantalla se prueba poco, solo para confirmar que todo funciona junto.



Niveles de prueba

Muestra sobre una arquitectura en capas (browser, web, negocio, datos) qué abarca cada nivel:

- **Unit:** un pedazo de código.
- **Integration:** varios componentes juntos.
- **System:** todas las capas.
- **Acceptance:** el flujo completo desde el browser.

Beneficios de automatizar

Rápida ejecución, re usabilidad, mayor cobertura, mayor precisión, mayor alcance y reducción de costos.

1. El testing exhaustivo es imposible
Un sistema de 20 pantallas da 20 × 3 × 4 × 10 × 100 = 240.000 pruebas, que serían 8,5 días a 1 seg por prueba y 13,7 años a 10 min. Hay que elegir qué probar según **riesgo y costo**, y usar métodos que cubran lo máximo con la menor cantidad de pruebas.

2. Pensar fuera de lo obvio
Es la lección del quiz de los 9 puntos: los buenos casos de prueba incluyen lo inesperado, como

valores fuera de rango, vacíos o combinaciones raras.

3. Caja negra
Se prueba **sin mirar el código**, solo comparando lo que entra con lo que sale según la especificación.

4. Los 4 métodos

- **Basados en especificaciones** (sistemáticos):
    - **Partición de equivalencias:** dividir las entradas en clases que el sistema trata igual y tomar un valor cualquiera de cada una.
    - **Valores límites:** variante de la partición que toma los **bordes** de cada clase, porque ahí se concentran los errores.
- **Basados en la experiencia** (dependen del tester):
    - **Adivinanza de defectos:** apostar dónde puede fallar según lo que se vio fallar antes.
    - **Testing exploratorio:** recorrer el sistema sin guion, aprendiendo y probando a la vez.

5. Partición en 2 pasos

1. Identificar las clases **válidas e inválidas**, según el tipo de entrada: rango continuo, valores discretos, selección simple o selección múltiple.
2. Identificar los **casos de prueba**: uno por clase.

6. Ningún método alcanza solo: hay que combinarlos.

Ejercicio de impuestos resuelto

Enunciado:

1. Hasta $4000, sin impuesto.
2. Los siguientes $1500, 10% del total.
3. Los próximos $2000, 22%.
4. Más que eso, 40% sobre el total.

Los tramos quedan así: 0–4000 · 4001–5500 · 5501–7500 · más de 7500. Tomo el porcentaje sobre el total percibido.

Paso 1: clases de equivalencia (entrada: monto percibido, rango continuo)

Clase	Condición	Tipo
CV1	$0 \leq \text{monto} \leq 4000$	Válida
CV2	$4001 \leq \text{monto} \leq 5500$	Válida

Clase	Condición	Tipo
CV3	5501 ≤ monto ≤ 7500	Válida
CV4	monto > 7500	Válida
CI1	monto < 0	Inválida
CI2	no numérico	Inválida

Paso 2: casos de prueba (partición)

Caso	Clase	Entrada	Resultado esperado
CP1	CV1	2000	Impuesto $0
CP2	CV2	5000	Impuesto $500
CP3	CV3	6000	Impuesto $1320
CP4	CV4	10000	Impuesto $4000
CP5	CI1	-100	Error: monto inválido
CP6	CI2	“abc”	Error: monto inválido

Valores límites

Caso	Entrada	Resultado esperado
VL1	-1	Error
VL2	0	$0
VL3	4000	$0
VL4	4001	$400,10
VL5	5500	$550
VL6	5501	$1210,22
VL7	7500	$1650
VL8	7501	$3000,40

QUE ES EL TESTING?

ISTQB: la más amplia
Para ISTQB, el testing es un proceso que acompaña **todo el ciclo de vida** del software, no algo que pasa al final. Incluye dos tipos de actividades, ESTATICAS Y DINAMICAS. Tampoco se limita a correr pruebas. También abarca **planificar** (qué y cómo probar) y **preparar** (casos y datos de prueba). Tiene tres objetivos: comprobar que cumple los requerimientos, demostrar que sirve para lo que se hizo y detectar defectos.

IEEE 610: la más operativa
Para la IEEE 610, testing es **ejecutar** el sistema o un componente bajo condiciones definidas, por ejemplo "con este usuario y estos datos", **observar y registrar** qué pasa y después **evaluar** algún aspecto del resultado. Es una visión puramente **dinámica**: si no se ejecuta el sistema, no es testing. Por eso deja afuera las revisiones de documentos y la planificación, que ISTQB sí incluye.

IEEE 829: la más cercana a la cátedra
Para la IEEE 829, testing es **analizar** el software para encontrar las **diferencias entre lo que existe y lo que se requería**, y a esas diferencias las llama directamente **bugs**. El foco está en **encontrar defectos** comparando el producto contra los requerimientos. Coincide con la visión de la cátedra: el testing es destructivo y es exitoso cuando encuentra defectos.Según la catedra:

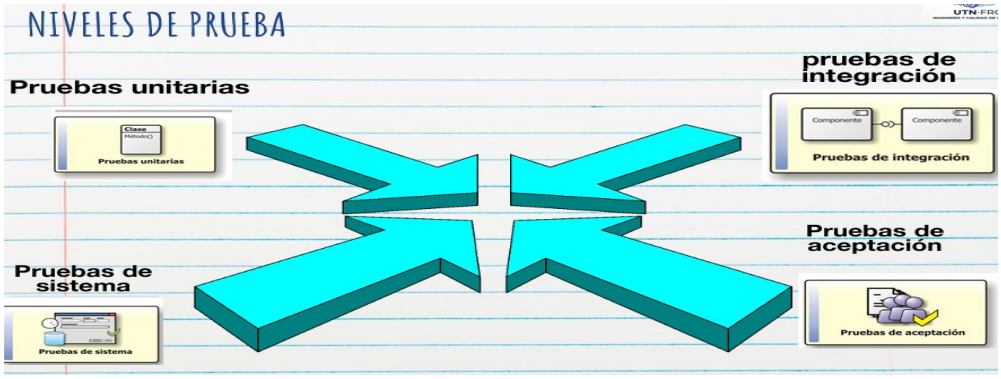
Es un proceso **destructivo** de tratar de encontrar defectos, cuya presencia **se asume**.

- Hay que ir con **actitud negativa**, para demostrar que algo está mal.
- **Testing exitoso = el que encuentra defectos.** Si no encontró nada, no fue exitoso.
- Cuesta entre el **30 y el 50%** del costo de un software confiable.

Asegurar la calidad vs controlar la calidad

- La calidad **no se "inyecta" al final**.
- Depende de lo que se hace **durante todo el proceso**.
- Detectar errores **temprano** ahorra esfuerzo, tiempo y recursos.
- La calidad abarca el **producto y el proceso**: mejorar el proceso evita defectos que se repiten.
- **El testing NO puede asegurar calidad.** Solo la controla, porque encuentra defectos en el producto. Asegurar es trabajar sobre el proceso.

ERROR → DEFECTO → FALLA .



1. Testing unitario: prueba de cada componente por separado, apenas se construye. La hace el developer con acceso al código y un framework de pruebas, y corrige los errores en el momento sin registro oficial. *Ejemplo:* verificar que `calcularSaldo(10000, 3000)` devuelve 7000.

2. Testing de integración: prueba de que los componentes que funcionan bien por separado también funcionan juntos.
Ejemplo: verificar que el módulo Extracción, al pedirle al módulo Cuentas que descuente $3000, deja el saldo en $7000 en la base de datos.

3. Testing de sistema: prueba de la aplicación completa funcionando como un todo, en un entorno lo más parecido posible a producción. Evalúa requerimientos funcionales y no funcionales (performance, seguridad, stress, recuperación de fallas).
Ejemplo: hacer una extracción completa desde el cajero, y comprobar que con 500 usuarios simultáneos responde en menos de 3 segundos.

4. Testing de aceptación: prueba que realiza el usuario para determinar si la aplicación se ajusta a sus necesidades. Su meta es dar confianza, no encontrar defectos. Puede ser alfa (en un ambiente de laboratorio) o beta (en el ambiente de trabajo real).
Ejemplo: el banco usa el cajero en una sucursal real y confirma que el proceso le sirve.

AMBIENTES PARA LA CONSTRUCCION DE SOFTWARE:

1. **Desarrollo:** donde los developers programan y hacen las pruebas unitarias.
2. **Prueba:** donde los testers ejecutan los casos de prueba, separado de desarrollo.
3. **Pre-producción:** una copia lo más fiel posible a producción. Ahí se hacen las pruebas finales, como las de sistema o aceptación, antes de liberar.
4. **Producción: donde el sistema lo usan los usuarios reales.**

Unitario vs integración

En el unitario cada componente se prueba **aislado**: si `calcularSaldo` depende de otro módulo, ese módulo se reemplaza por uno falso, así el test solo mira esa pieza. Lo hace el developer con acceso al código, y si falla, arregla en el momento sin dejar registro.

En integración se sacan esos reemplazos y se conectan las piezas reales. Una pieza puede andar perfecto sola y fallar al conectarse: por ejemplo, un módulo manda el monto en pesos y el otro lo espera en centavos. Esos errores de **interfaz entre componentes** solo aparecen en integración. Por eso se integra **de a poco** (top-down, bottom-up o combinados): si conectás todo de golpe y falla, no sabés dónde está el problema.

Integración vs sistema

Integración todavía mira **partes**: algunos módulos conectados entre sí. Sistema mira el **producto entero**, como lo va a recibir el cliente. Hay dos cosas que solo aparecen en sistema:

1. **Lo no funcional:** performance, carga, stress, seguridad y recuperación de fallas solo tienen sentido con el sistema completo funcionando.
2. **El entorno:** se prueba en un ambiente **lo más parecido a producción**, porque hay errores que dependen del ambiente (servidor, red, configuración) y no aparecen en la máquina del developer.

Sistema vs aceptación

Los dos prueban el sistema completo. La diferencia es **quién y para qué**:

- **Sistema:** lo hace el **equipo**, con actitud destructiva. Busca **encontrar defectos** contra los requerimientos especificados.
- **Aceptación:** la hace el **usuario**. No busca romper el sistema sino confirmar que **le sirve para su trabajo**, y así **ganar confianza** para aceptarlo. Un sistema puede pasar todas las pruebas de sistema y ser rechazado en aceptación, porque cumple lo escrito pero no lo que el usuario realmente necesitaba.

Idea que une todo: cada nivel encuentra errores que el anterior no puede ver.

- **Unitario:** errores dentro de una pieza.
- **Integración:** errores entre piezas.
- **Sistema:** errores del producto completo y del ambiente.
- **Aceptación:** errores de “esto no es lo que necesitaba”.

CASOS DE PRUEBA

Es **una prueba concreta**, con objetivo, datos de entrada y resultado esperado. Es la unidad mínima del testing.

Relación con NIVELES DE TESTING (SON LOS EJEMPLOS MENCIONADOS): en **cada nivel se ejecutan casos de prueba**. El nivel indica el alcance y los casos son las pruebas concretas que se hacen dentro de ese alcance.

OBJETIVO	**descubrir errores**
CRITERIO	**en forma completa**
RESTRICCIÓN	**con el mínimo de esfuerzo y tiempo**

"Los bugs se esconden en las esquinas y se congregan en los límites … "

Derivación de casos de prueba

De dónde salen los casos, en una escalera que va de lo más cercano al cliente a lo más técnico:

1. Desde documentos del cliente.
2. Desde información de relevamiento.
3. Desde requerimientos.
4. Desde especificaciones de programación.
5. Desde el código.

1. Caso de prueba: una prueba individual.

2. Smoke test: una prueba rápida y básica que se corre **antes** del ciclo, para confirmar que el sistema arranca y no tiene una falla catastrófica. Si falla, no tiene sentido correr los demás casos y la versión vuelve a desarrollo.

3. Ciclo de test: una ejecución de todos los casos sobre una misma versión. Agrupa los casos.

4. Regresión: volver a ejecutar el ciclo completo sobre una nueva versión, para ver que los arreglos no rompieron nada. Es la repetición del ciclo.

Smoke test → filtro rápido antes de cada ciclo o regresión

Regresión → repetir el ciclo en la versión nueva

└ Ciclo de test → correr todos los casos en una versión

└ Caso de prueba → una prueba individual

Ejemplo:

- **Caso:** "extraer $5000 con saldo $2000 → Saldo insuficiente".
- **Smoke test:** llega la versión 1.0 y se verifica que el cajero abre, loguea, y hace 1 extraccion.
- **Ciclo:** como el smoke test pasó, se corren los 50 casos sobre la versión 1.0.
- **Regresión:** se corrigen los bugs, sale la 1.1, se hace el smoke test y se corren de nuevo los 50 casos sobre la 1.1.

Cómo se relacionan:

1. Llega una versión → **smoke test**: ¿vale la pena probarla?
2. Si pasa → **ciclo**: se corren todos los casos.

3. Se corrigen bugs y llega una versión nueva → **smoke test** de nuevo y después la **regresión**: se repite el ciclo completo.

***ESTRATEGIAS DEL TESTING*:**

- **El tiempo y el presupuesto es limitado.**
- **Hay que pasar por la mayor cantidad de funcionalidades con la menor cantidad de pruebas**

Caja negra: se diseñan los casos **sin mirar el código**, solo a partir de la **especificación**: qué entra y qué debería salir. El sistema es una caja cerrada.
Métodos: partición de equivalencias, valores límites, adivinanza de defectos y testing exploratorio.

Caja blanca: se diseñan los casos **mirando la estructura interna del código**: sus caminos, decisiones y sentencias. El sistema es una caja abierta.
Métodos: cobertura de caminos básicos, de sentencias, de decisión, de condición, de decisión/condición y múltiple.

Diferencia clave:

- **Caja negra** prueba **qué hace** el software (si cumple lo especificado) y **no garantiza cobertura** del código. Caja negra puede hacerla alguien que no programa.
- **Caja blanca** prueba **cómo lo hace** (si todo el código se ejecuta y se comporta bien) y **sí garantiza cobertura**, porque se puede medir qué porcentaje del código se recorrió. Caja blanca requiere acceso al código y saber leerlo.

1. Planificación y Control
Planificar es entender los objetivos del cliente, los stakeholders y los riesgos, y armar el Test Plan: riesgos y objetivos, estrategia, recursos y criterio de aceptación. Controlar es revisar los resultados y la cobertura durante el proceso y tomar decisiones.
Entregable: plan de testing.
Diferencia: planificar es *antes* (qué y cómo probar) y controlar es *durante* (cómo vamos).

2. Identificación y Especificación
Se revisa de dónde salen las pruebas (requerimientos y especificaciones), se evalúa si se pueden probar y se diseñan y priorizan los casos, los datos y el entorno.
Entregable: documento de casos de prueba.

3. Ejecución
Se crean los datos, se automatiza lo necesario, se prepara el ambiente, se corren los casos y se registra el resultado junto con la versión probada. Al final se compara lo real con lo esperado.
Entregable: reporte de incidentes, donde quedan registrados los defectos encontrados.
Diferencia con la etapa 2: en la 2 se *diseñan* los casos y en la 3 se *ejecutan*.

4. Evaluación y Reporte
Se evalúa si se cumplió el criterio de aceptación, se informa a los stakeholders, se verifica que los defectos estén corregidos y se registran las lecciones aprendidas.
Entregable: informe final.
Diferencia con Control: el control *revisa durante* el proceso y la evaluación *cierra al final* con el informe.

CICLO DE VIDA DEL TESTING

La idea es que el testing acompaña todo el ciclo de vida del software, desde el análisis de necesidades hasta la construcción, y no solo el final.

Verificación vs validación

Las dos controlan el producto, pero lo comparan contra cosas distintas.

- **Verificación:** ¿estamos construyendo el sistema *correctamente*? Compara el producto contra la **especificación**, o sea, si se hizo bien lo que estaba escrito.
- **Validación:** ¿estamos construyendo el sistema *correcto*? Compara el producto contra la **necesidad real del usuario**, o sea, si lo que se hizo es lo que el usuario quería.

Diferencia: un sistema puede estar perfectamente verificado (cumple todo lo especificado) y fallar en la validación, porque la especificación no reflejaba lo que el usuario necesitaba.

Testing temprano en el ciclo de vida

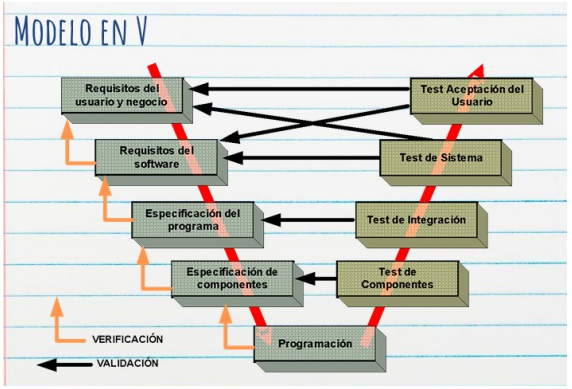
Hay dos objetivos para involucrar el testing desde el inicio:

- Que el equipo sepa desde temprano **cómo se va a probar** el producto
- **Bajar el costo** de corregir defectos, porque un error detectado en requerimientos sale mucho más barato que uno detectado en producción.

Modelo en V

Une el ciclo de vida con el testing. El lado izquierdo baja construyendo y el lado derecho sube probando. Cada nivel de prueba se compara con la etapa que tiene a su misma altura:

- Requisitos de usuario ↔ test de aceptación.
- Requisitos de software ↔ test de sistema.
- Especificación del programa ↔ test de integración.
- Especificación de componentes ↔ test de componentes.
- En el vértice, la programación.



Diferencia dentro del modelo: las flechas que bajan por el lado izquierdo (cada etapa contra la anterior) son **verificación**. Las flechas que van de derecha a izquierda (cada prueba contra su etapa) son **validación**. Así la V muestra las dos cosas: construir bien y construir lo correcto.

Mitos

El testing **no** empieza al terminar de codificar, **no** es probar que el software funciona, **no** es igual a calidad de producto ni de proceso, y el tester **no** es enemigo del programador.

¿Cuánto testing es suficiente?

Como probar todo es imposible, el **riesgo** define *qué* probar y el **criterio de aceptación** define *cuándo parar*. Decidir cuánto testing es suficiente depende de:

- Evaluación del nivel de riesgo.
- Costos asociados al proyecto.
- Fallas predichas aún permanecen en el software

Los 7 principios

1. **Muestra presencia de defectos:** puede probar que hay errores, nunca que no los hay.
2. **El exhaustivo es imposible:** no se puede probar todo.
3. **Testing temprano:** cuanto antes se prueba, más barato es corregir.
4. **Agrupamiento de defectos:** la mayoría de los defectos se concentran en pocos módulos.
5. **Paradoja del pesticida:** renovar pruebas, sino las mismas siempre no sirven.
6. **Depende del contexto:** no se prueba igual un juego que un sistema bancario.
7. **Falacia de la ausencia de errores:** que no tenga errores no significa que sirva al usuario.

TIPOS DE PRUEBAS:

Funcional: prueba QUÉ hace el sistema
Verifica que cada función haga lo que dicen los requerimientos o el proceso de negocio.
Ejemplo: extraer $3000 entrega $3000 y descuenta el saldo.

- **Basada en requerimientos:** se prueba una regla puntual.
  Ejemplo: según el requerimiento "si el saldo no alcanza, se rechaza", con saldo $2000 se intenta extraer $5000 y debe aparecer "Saldo insuficiente".
- **Basada en procesos de negocio:** se prueba el flujo completo del usuario.
  Ejemplo: insertar tarjeta → PIN → monto → recibir dinero y comprobante → saldo actualizado.

No funcional: prueba CÓMO lo hace
El sistema puede hacer todo bien y aun así ser lento, caerse o ser difícil de usar. La slide insiste en que no hay que olvidarlas. Las no funcionales del PDF:

- **Performance:** qué tan rápido responde. *¿La extracción tarda menos de 3 segundos?*
- **Carga:** cómo responde con la cantidad de usuarios esperada. *500 clientes extrayendo.*
- **Stress:** qué pasa cuando se lo lleva más allá del límite. *¿se cae o se degrada?*
- **Volumen:** cómo maneja grandes cantidades de datos.
- **Usabilidad:** qué tan fácil es de usar. *¿Un jubilado entiende los mensajes?*
- **Fiabilidad:** que funcione sin fallar en el tiempo y se recupere si falla. Por ej la luz.
- **Mantenimiento:** qué tan fácil es de modificar o corregir.
- **Portabilidad y configuración:** que funcione en distintos entornos. *Distintos modelos de cajero, sistemas operativos o redes.*
- **Instalación:** que se instale y actualice correctamente.

LEAN KANBAN

El objetivo de todo esto es uno solo: **entregar valor al cliente lo más rápido posible, sin desperdiciar esfuerzo**.

Lean y Kanban no son lo mismo:

	Lean	Kanban
Qué es	Una **filosofía**, una forma de pensar	Un **método** concreto para trabajar
Qué dice	"Eliminá todo lo que no le agrega valor al cliente"	"Así se organiza el trabajo para lograrlo"
Herramientas	Principios y desperdicios	Tablero, límites de WIP, políticas, métricas

Lean dice qué buscar, y Kanban es una forma de llevarlo a la práctica.

Para qué sirve en software: un equipo puede estar muy ocupado y aun así entregar poco, porque empieza muchas cosas y no termina ninguna, espera aprobaciones, cambia de tarea todo el tiempo o hace funcionalidades que nadie pidió. Eso son los desperdicios. Kanban los hace **visibles** en el tablero y los **controla** limitando el WIP, para que el trabajo fluya y se termine.

}Scrum y Kanban son dos formas ágiles de gestionar el trabajo, con enfoques distintos.

	Scrum	Kanban
Ritmo	Sprints de duración fija	Flujo continuo, sin iteraciones
Qué limita	Lo que entra en el sprint	Cuántos ítems hay en cada columna (WIP)
Roles	Definidos (PO, SM, equipo)	Respeta los roles que ya existen
Cambio	Se adopta el marco completo	Empieza con lo que hacés hoy y mejora de a poco

"¿para qué sirve Kanban?"**para visualizar y limitar el trabajo en curso, de modo que fluya y se entregue valor al cliente de forma continua y predecible, eliminando desperdicios.**

PRINCIPIOS DE KANBAN

Gestión de cambios trata de cómo se introduce Kanban en un equipo. Se empieza con lo que ya se hace, entendiendo el proceso actual y respetando los roles, responsabilidades y puestos existentes. A partir de ahí se acuerda mejorar de forma gradual, con cambios evolutivos y no de golpe, y se fomenta que cualquiera, en cualquier nivel, pueda liderar una mejora.

Entrega de servicios trata de **cómo se trabaja una vez que Kanban está aplicado**. El foco está en cumplir las necesidades y expectativas del cliente, se gestiona el trabajo (no a las personas) dejando que el equipo se auto organice, y se revisan periódicamente los servicios y sus políticas para mejorar los resultados.

La diferencia: el primer grupo es sobre **adoptar Kanban sin romper lo que existe**, y el segundo sobre **entregar valor al cliente una vez adoptado**.

PRACTICAS GENERALES

1. **Visualizar:** mostrar en un tablero el trabajo, su flujo y sus riesgos, para que todos vean lo mismo y se decida más rápido.
2. **Limitar el trabajo en curso (WIP):** poner un máximo de tareas por etapa según la capacidad real. “Dejá de empezar, empezá a terminar.”
3. **Gestionar el flujo:** hacer que el trabajo avance de forma continua, predecible y confiable, usando datos para controlarlo.
4. **Hacer las políticas explícitas:** que las reglas de trabajo estén acordadas y visibles para todos (cuándo se toma una tarea, los límites de WIP, las clases de servicio).
5. **Establecer ciclos de retroalimentación:** reuniones con una frecuencia definida (las cadencias) para revisar, aprender y mejorar.
6. **Mejorar colaborativamente y evolucionar experimentalmente:** probar cambios como experimentos seguros. Si funcionan se mantienen, y si no, se vuelve atrás.

Lean es la **filosofía** detrás de Kanban. Su objetivo es **maximizar el valor para el cliente usando la menor cantidad de recursos**, eliminando todo lo que no le aporta valor (los desperdicios).

Origen: viene de la manufactura, del sistema de producción de Toyota. Después se adaptó al software, y por eso la tabla traduce cada desperdicio de fábrica a su equivalente en desarrollo.

La idea central: todo lo que se hace en un proceso es **valor** (algo por lo que el cliente pagaría) o **desperdicio**. Lean busca que haya cada vez más de lo primero y menos de lo segundo:

- Al **puro desperdicio** se lo **evita** (características extra que nadie pidió, trabajo a medio terminar).
- Al **desperdicio necesario** se lo **minimiza** (cosas que no agregan valor directo pero hay que hacer, como cierta documentación o reuniones).

Los 7 principios son las reglas para lograr eso:

- Eliminar desperdicios.
- Aprender rápido y a partir de feedback.
- Decidir lo más tarde posible, con más información.
- Entregar rápido.
- Darle poder al equipo.
- Construir con calidad desde el inicio (integridad).
- Mirar el proceso completo y no solo una parte.

MÉTRICAS DE SOFTWARE EN LOS DIFERENTES ENFOQUES DE GESTIÓN

Cada enfoque de gestión mide cosas distintas porque gestiona de forma distinta.

Tradicional (procesos definidos): mide cuánto se esfuerza y gasta el proyecto, comparando lo planificado contra lo real. Las métricas se dividen en de proceso, de proyecto y de producto. Sirven para balancear las expectativas de los involucrados con alcance, tiempo, recursos y calidad. La regla es mantenerlo simple: si estás lejos del objetivo, no tiene sentido medir con precisión.

- **Tamaño** del producto: cuánto software se construye.
- **Esfuerzo**: horas de trabajo invertidas.
- **Tiempo** (calendario): cuánto dura el proyecto o una tarea.
- **Defectos**: cuántos errores se encuentran y dónde.

El desarrollador mide el **esfuerzo** y la duración de sus tareas, la cobertura de su unit test y los defectos que encuentra en unit test y revisión por pares. El **equipo** mide tamaño, tareas planificadas contra completadas, estado y porcentaje de tests ejecutados. La **organización** mide tiempo calendario, esfuerzo, presupuesto real contra planificado y **defectos** en el release.

Ágil (procesos empíricos): mide cuánto valor funcionando se entrega. La regla de oro es que la medición es una salida, no una actividad: se obtiene del trabajo y no es una tarea extra. Se guía por dos principios ágiles: la prioridad es satisfacer al cliente con entregas tempranas y continuas.

- **Velocidad:** puntos de historia que el equipo termina por sprint. Solo historias terminadas.
- **Capacidad:** cuánto puede trabajar el equipo en un sprint, en horas o puntos de historia.
- **RTF (Running Tested Features)**: funcionalidades funcionando y testeadas. Es la medida directa de “software funcionando”.

Lean (Kanban): mide qué tan rápido fluye el trabajo desde que se pide hasta que se entrega, y cuánto tiempo se pierde esperando.

- **Lead Time:** desde que el cliente pide un ítem hasta que se le entrega. Es la vista del cliente, se mide en días de trabajo y muestra el ritmo de entrega.
- **Cycle Time:** desde que se empieza a trabajar el ítem hasta que termina. Es la vista interna, mide la capacidad del proceso y muestra el ritmo de terminación.
- **Touch Time:** solo los días en que el ítem se trabajó de verdad, trabajo en curso.
- **Eficiencia del proceso:** Touch Time / tiempo total transcurrido. Cuanto más alta, mejor.

Métricas orientadas a servicio:

- **Expectativa:** el nivel de servicio que el cliente espera.
- **Capacidad:** el nivel que el sistema puede entregar.
- **Acuerdo:** el nivel pactado con el cliente.
- **Umbral:** el nivel por debajo del cual el servicio es inaceptable.

Diferencia: tradicional mide el esfuerzo y el gasto contra lo planificado, ágil mide el valor funcionando que se entrega, Lean mide el flujo y los tiempos del trabajo, y producto mide el software en sí.